

PYTHON WORKSHOP

Vesmírné Crossy Road

2D top-down hra v Pythonu + Pygame

Autor: Bc. Stefan Rajilić

Předmět: Python Workshop – Karel Šafr, Ph.D.

Rok: 2025/2026

Jazyk: Python 3 | Knihovna: pygame

Soubor: main.py

Obsah

1. Úvod a herní mechanika
2. Technické prostředí a spuštění
3. Architektura a struktura kódu
4. Třídy spritů – Lod, Prekazka, Vybuch
5. Přepočet souřadnic: mřížka → pixely
6. Herní smyčka – EVENT / UPDATE / RENDER
7. Systém levelů a obtížnosti
8. Vizuální efekty
9. High-score a persistence dat
10. Grafické assety
11. Co by šlo ještě zlepšit

1 Úvod a herní mechanika

Hra Vesmírné Crossy Road je inspirovaná mobilní hrou Crossy Road – základní mechanika je jednoduchá, ale implementace má spoustu zajímavých technických aspektů: pohyb po mřížce, plynulý pohyb překážek přes float souřadnice, detekce kolizí přes sprite skupiny, správa stavů hry a několik vizuálních efektů.

Téma jsem zasadil do vesmíru. Hráč ovládá vesmírnou loď a musí proletět přes asteroidy a trosky satelitů, dostat se ze spodního okraje obrazovky na horní, a to opakovaně – každé dokončení trasy přináší nový, těžší level.

Jak se hraje

- Loď se ovládá **šipkami** – každý stisk = jeden krok v mřížce.

- Na řádcích 1–6 a 8–13 se pohybují překážky (meteority nebo trosky satelitů).
- Řádek 7 je **bezpečný ostrůvek** – zde si hráč může odpočinout.
- Srážka s překážkou = ztráta života. Hráč začíná se **3 životy**.
- Po zásahu je hráč po dobu **1,5 sekundy** neporazitelný (grace period) a loď bliká.
- Za každý krok nahoru hráč získá **1 bod**. Za dosažení horního řádku **+50 bodů**.
- Po dosažení cíle se loď resetuje na start a spustí se 1,5sekundová přechodová animace na nový level.
- Při nule životů se zobrazí obrazovka Game Over a rekord se uloží do souboru.

Stavový diagram hry

```

SPLASH (5 s nebo libovolná klávesa/klik) | ▼ MENU
      ────────────────────────────────────> SETTINGS (výběr lodi) | | | ENTER "Hrát
hru" | ENTER / ESC | ┌───────────────────> MENU ▼ PLAYING ─── dosažení cíle
      ────────────────────────────────────> LEVEL_UP (1,5 s animace) | | | ┌───────────────────> PLAYING (nový
level) | životy = 0 ▼ GAMEOVER ─── ENTER ───────────────────> MENU

```

2 Technické prostředí a spuštění

Parametr	Hodnota
Jazyk	Python 3.10+
Externí knihovny	pygame 2.6+ (jedinou povinnou závislostí)
Rozlišení okna	800 × 600 px
FPS	60 snímků/s
Velikost buňky mřížky	40 × 40 px (konstanta <code>TILE</code>)
Mřížka	20 sloupců × 15 řádků (<code>COLS</code> × <code>ROWS</code>)
Startovní pozice lodi	sloupec 10, řádek 14 (střed dolní řady)

Instalace a spuštění

```
# 1. Vytvoř a aktivuj virtuální prostředí
python3 -m venv .venv
source .venv/bin/activate      # macOS / Linux
.venv\Scripts\activate        # Windows

# 2. Nainstaluj závislosti
pip install pygame

# 3. Spustí hru
python3 main.py
```

Hra funguje i bez assetů. Chybějící obrázky se automaticky nahradí fialovým placeholderem. Pro plnou grafiku viz sekci 10.

3 Architektura a struktura kódu

Celý kód je v jednom souboru `main.py`. Pro rozsah téhle hry je to úplně dostačující – zbytečné dělení do modulů by jen přidávalo složitost bez reálného přínosu. Struktura je shora dolů:

main.py |— Konstanty (barvy, rozměry, FPS, jména souborů, definice pruhů PRUHY) |— pygame.init() + display + clock + ASSET_DIR |— Pomocný loader: nacti_obrazek() |— Fonty |— High-score: nacti_highscore(), uloz_highscore() |— Třídy spritů | |— Lod - hráčova loď, pohyb po mřížce | |— Prekazka - letící překážky (meteority / trosky satelitů) | |— Vybuch - animovaný efekt výbuchu při zásahu |— Parallax hvězdy: HVEZDY_FAR, HVEZDY_NEAR, kresli_hvezdy() |— Pomocné funkce vykreslování | |— vytvor_prekazky() - sestavení skupiny překážek pro daný level | |— vykresli_pozadi() - pruhy drah + ostrůvek + hvězdy | |— vykresli_trysky() - animovaný plamen pod lodí | |— vykresli_hud() - skóre, rekord, životy, level | |— vykresli_text_center() | |— vykresli_tlacitko() - menu tlačítka |— Předběžné načtení assetů (lodě, splash) |— Inicializace herních proměnných |— Hlavní smyčka while running: |— EVENT - zpracování fronty událostí |— UPDATE - herní logika, pohyb, kolize |— RENDER - vykreslení snímku podle aktuálního stavu

Klíčové konstanty

Konstanta	Hodnota	Popis
TILE	40	Velikost jedné buňky mřížky v px
COLS / ROWS	20 / 15	Počet sloupců a řádků
START_ROW	14	Startovní řádek lodi (dolní okraj)
OSTROV_RADEK	7	Řádek bezpečného ostrůvku – bez překážek
GRACE_PERIOD	1 500 ms	Neporanitelnost po zásahu
SPLASH_TRVANI	5 000 ms	Délka úvodní splash obrazovky (5 s)
ZIVOTY_START	3	Počáteční počet životů

Definice pruhů (PRUHY)

Každý prvek seznamu PRUHY definuje jednu vesmírnou dráhu. Řádek 7 (OSTROV_RADEK) záměrně chybí – je to bezpečná zóna.

```

PRUHY = [
    (řádek, základní_rychlost, typ, počet),
    # ↑         ↑           ↑       ↑
    # index     px/snímek   0=meteor  výchozí počet
    # mřížky    záporná=←   1=trosky  objektů v pruhu
    ...
]

```

Rychlost je základní – za běhu se násobí hodnotou `speed_mult`, která roste s každým levellem.

4 Třídy spritů

Všechny herní objekty dědí z `pygame.sprite.Sprite`. Díky tomu je spravuju přes `pygame.sprite.Group` – hromadné volání `update()` a `draw()` je pak na jeden řádek, což odpovídá vzoru z přednášek pana Šafra.

Třída Lod

Reprezentuje loď hráče. Pozice se ukládá jako logické souřadnice mřížky (`grid_col`, `grid_row`), ne jako pixely. Pixelové souřadnice rectu se přepočítávají metodou `_sync_rect()` pokaždé, kdy se loď pohne nebo resetuje. Hráč si může vybrat ze čtyř lodí – obrázky se načítají jednou do třídního atributu `OBRAZKY` a sdílí se mezi instancemi.

Metoda / atribut	Co dělá
<code>OBRAZKY</code> , <code>NAHLEDKY</code>	Třídní atributy – obrázky lodí sdílené mezi instancemi, načtou se jednou před spuštěním smyčky
<code>nacti_vsechny()</code>	Classmethod – načte 4 obrázky lodí; při opakovaném volání nic nedělá (guard <code>if cls.OBRAZKY: return</code>)
<code>_sync_rect()</code>	Přepočítá <code>rect.x</code> / <code>rect.y</code> z mřížkových souřadnic (vzorec viz sekce 5)
<code>reset()</code>	Vrátí loď na startovní pozici (sloupec 10, řádek 14)
<code>move(d_col,</code> <code>d_row)</code>	Posune loď o krok, zastaví na okrajích; vrátí (<code>pohnul_se</code> , <code>dosahl_cile</code>)
<code>update()</code>	Prázdná – pohyb řídí KEYDOWN eventy, ne update smyčka

Třída Prekazka

Pohybující se překážka – meteorit nebo trosky satelitu. Pohyb je plynulý (pixels), na rozdíl od lodí, která skáče po mřížce. Aby nedocházelo k trhavému pohybu při nízkých rychlostech (např. 1.5 px/snímek), ukládám přesnou polohu jako `float pos_x` a do `rect.x` přiřazuju teprve zaokrouhlenou hodnotu.

Obrázky sdílí třídní slovník `_cache` – stejný soubor se nenačítá vícekrát, jen se vrátí existující Surface.

Metoda / atribut	Co dělá
<code>_cache</code>	Třídní slovník – klíč je (název, šířka, výška) ; zabraňuje opakovanému načítání stejné textury
<code>_ziskej()</code>	Vrátí Surface z cache nebo ho načte a uloží
<code>pos_x</code>	Přesná float X-souřadnice pro plynulý pohyb
<code>update(speed_mult)</code>	Pohne překážkou o <code>rychlost × speed_mult</code> px; po opuštění obrazovky ji zacyklí na opačné straně s náhodným offsetem

Třída Vybuch

Animovaný efekt výbuchu, který se zobrazí při zásahu lodi. Třída načítá snímky `vybuch_01.png` až `vybuch_07.png` a přehrává je po 70 ms každý. Celá animace tak trvá přibližně $7 \times 70 \text{ ms} = 490 \text{ ms}$. Pokud snímky neexistují, použije se fallback na jediný obrázek `vybuch.png`. Po přehrání posledního snímku se sprite sám odstraní metodou `kill()`.

Snímky se načítají do třídního atributu `FRAMES` – jednou při prvním výbuchu (lazy loading) a pak se sdílí mezi všemi instancemi.

```
def update(self):
    nyní = pygame.time.get_ticks()
    if nyní - self.cas_snimku ≥ self.FRAME_TRVANI:    # 70 ms
        self.snimek += 1
        self.cas_snimku = nyní
        if self.snimek ≥ len(Vybuch.FRAMES):
            self.kill()    # odstraní sprite ze všech skupin
            return
        self.image = Vybuch.FRAMES[self.snimek]
```

5 Přepočet souřadnic: mřížka → pixely

Celá obrazovka je rozdělena na buňky `TILE × TILE` ($40 \times 40 \text{ px}$). Loď má Surface $36 \times 36 \text{ px}$ (`TILE - 4`), aby byl okolo vidět malý okraj. Pixelové souřadnice rectu se počítají v `_sync_rect()`:

```
def _sync_rect(self):
    self.rect.x = self.grid_col * TILE + 3
    self.rect.y = self.grid_row * TILE + 3

# Příklad: loď na sloupci 10, řádku 14
# rect.x = 10 × 40 + 3 = 403 px
# rect.y = 14 × 40 + 3 = 563 px
# Surface 36 px se vejde do buňky 40 px s 2px mezerou na každé straně.
```

Podobný princip platí pro překážky – jejich svislá poloha se nastavuje přes `rect.centery`, aby byly přesně uprostřed svého pruhu:


```
# Střed překážky = střed příslušného řádku mřížky
self.rect.centery = radek * TILE + TILE // 2
# Vzorec: index_řádku × 40 + 20 = střed buňky v pixelech
# Příklad řádek 3: 3 × 40 + 20 = 140 px od horního okraje
```

6 Herní smyčka – EVENT / UPDATE / RENDER

Celá logika hry běží v jedné smyčce `while running:`, která se opakuje 60× za sekundu. Smyčka je rozdělena na tři striktně oddělené části podle vzoru z přednášek.

EVENT – zpracování vstupů

Čtu frontu událostí přes `pygame.event.get()`. Stejná klávesa dělá různé věci podle aktuálního stavu (`stav`): šipky v menu procházejí položky, ve hře pohybují lodí. Pohyb lodi funguje přes `KEYDOWN` – jedno stisknutí generuje přesně jednu událost, takže loď se pohne o přesně jeden krok v mřížce.

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        uloz_highscore(skore)
        running = False
    elif event.type == pygame.KEYDOWN:
        if stav == PLAYING:
            if event.key == pygame.K_UP:
                pohyb = (0, -1) # krok nahoru = řádek - 1
                ...
            pohnu_lse, dosahl_cile = lod.move(*pohyb)
            if dosahl_cile:
                stav = LEVEL_UP # → přechodová animace
```

UPDATE – herní logika

Tady se dějí věci, které se aktualizují každý snímek bez ohledu na vstup. Parallax hvězdy scrollují vždy, i v menu:

```

# Vždy (ve všech stavech) - dávají pocit pohybu vesmírem
par_far_y = (par_far_y + 0.15) % HEIGHT
par_near_y = (par_near_y + 0.40) % HEIGHT

# Jen ve stavu PLAYING:
prekazky.update(speed_mult) # pohyb všech překážek najednou
hrac_grp.update()
vybuch_grp.update()         # Vybuch se sám odstraní po přehrání
snímků

# Detekce kolize - jen mimo grace period (1 500 ms po posledním
zásahu)
if nyní - cas_zasahu > GRACE_PERIOD:
    zasahy = pygame.sprite.spritecollide(lod, prekazky, False)
    if zasahy:
        zivoty -= 1
        cas_zasahu = nyní
        vybuch_grp.add(Vybuch(lod.rect.centerx, lod.rect.centery))
        lod.reset()
        if zivoty ≤ 0:
            uloz_highscore(skore)
            stav = GAMEOVER

```

RENDER – vykreslování

Vykreslování probíhá v pevném pořadí – co chci mít vizuálně „nahore“, kreslím jako poslední. Ve stavu **PLAYING** :

1. `vykresli_pozadi()` – barevné pruhy drah + ostrůvek + linky + parallax hvězdy
2. `prekazky.draw(screen)` – překážky
3. `vykresli_trysky(lod)` – animovaný plamen trysky (před lodí, aby byl vizuálně za ní)
4. `hrac_grp.draw(screen)` – loď
5. `vybuch_grp.draw(screen)` – efekty výbuchu
6. Blikání lodi při grace period (každých 100 ms)
7. `vykresli_hud()` – skóre, rekord, životy, level (vždy úplně nvrchu)
8. `pygame.display.flip()` – přehodí double-buffer, zobrazí snímek

Proč double-buffer? Pygame kreslí do zákulisního bufferu, který se zobrazí najednou přes `flip()`. Uživatel nikdy nevidí rozkreslený snímek – to je důvod, proč hra neblíká.

7 Systém levelů a obtížnosti

Obtížnost se zvyšuje dvěma způsoby zároveň – rychlostí překážek a jejich počtem.

Rychlost překážek (speed_mult)

Proměnná `speed_mult` začíná na 1.0 a při každém dosažení cíle se zvýší o 0.2. Předává se jako argument do `prekazky.update(speed_mult)`:

```
# Každá překážka pohyb = základní_rychlost × speed_mult
self.pos_x += self.rychlost * speed_mult

# Level 1: rychlost pruhu 2.5 × 1.0 = 2.5 px/snímek
# Level 5: rychlost pruhu 2.5 × 1.8 = 4.5 px/snímek (+80 % rychlost)
```

Počet překážek (vytvor_prekazky)

Při každém přechodu na nový level se celá skupina překážek přegeneruje s vyšším počtem objektů. Každé dva levely přibude jedna překážka v každém pruhu:

```
def vytvor_prekazky(level=1):
    bonus = (level - 1) // 2 # level 1→0, 2-3→1, 4-5→2, ...
    for radek, rychlost, typ, pocet in PRUHY:
        for _ in range(pocet + bonus):
            skupina.add(Prekazka(radek, rychlost, typ))
```

Level	speed_mult	Bonus překážek/pruh
1	1.0	+0
2	1.2	+0
3	1.4	+1
4	1.6	+1
5	1.8	+2
6+	2.0+	+2+

Přechodová animace level-up

Při dosažení horního okraje se stav změní na `LEVEL_UP`. Po dobu 1,5 sekundy se zobrazí bílý flash overlay, který postupně mizí, a text „LEVEL X! / Rychlost roste!“. Překážky jsou po tuto dobu pozastavené (jejich `update()` se nevolá). Po 1,5 sekundě se stav přepne zpět na `PLAYING` a hra pokračuje s novou obtížností.

8 Vizuální efekty

Parallax hvězdy

Hvězdy jsou rozděleny do dvou vrstev pro pocit hloubky. Obě vrstvy scrollují shora dolů s různou rychlostí – vzdálenější pomaleji, bližší rychleji. Scrollování probíhá každý snímek i v menu a na splash obrazovce.

```
# Vzdálená vrstva (70 hvězd, šedavé, poloměr 1 px):
par_far_y = (par_far_y + 0.15) % HEIGHT

# Bližká vrstva (30 hvězd, bílé, poloměr 2 px):
par_near_y = (par_near_y + 0.40) % HEIGHT

# Vykreslení: pozice Y se offsetuje a zacykluje přes modulo HEIGHT
dy = int((y + par_far_y) % HEIGHT)
pygame.draw.circle(screen, (165, 165, 200), (x, dy), 1)
```

Animované trysky lodí

Pod lodí se vykresluje plamen trysky se třemi střídajícími se snímky (každých 80 ms). Trysky se kreslí před voláním `hrac_grp.draw()`, takže jsou vizuálně za lodí. Plamen se skládá ze dvou soustředných elips – vnější oranžové a vnitřní žlutobílé.

```
frame = (pygame.time.get_ticks() // 80) % 3
vel = [10, 7, 13][frame] # výška plamene
sír = [7, 5, 9][frame] # šířka plamene
```

Animovaný výbuch

Třída `Vybuch` načítá snímky `vybuch_01.png` až `vybuch_07.png` ze složky `assets/`. Každý snímek se zobrazí 70 ms, celá animace trvá přibližně 490

ms. Pokud snímky neexistují, použije se jediný záložní obrázek `vybuch.png`. Po přehrání se sprite sám odstraní ze skupiny.

Bezpečný ostrůvek

Řádek 7 (`OSTROV_RADEK = 7`) je záměrně vynechán ze seznamu `PRUHY` – nemá žádné překážky. Vykresluje se tmavě zelenou barvou se zvýrazněnými linkami hranic, takže hráč ho jasně vidí jako bezpečnou zónu přibližně v polovině mapy.

9 High-score a persistence dat

Nejlepší skóre se ukládá do textového souboru `highscore.txt` umístěného vedle `main.py`. Při každém ukončení hry (Game Over, zavření okna, výběr „Ukončit hru“) se porovná aktuální skóre s uloženým rekordem a přepíše se jen v případě, že bylo překonáno.

```
HS_FILE = path.join(path.dirname(__file__), "highscore.txt")

def nacti_highscore():
    try:
        with open(HS_FILE, 'r') as f:
            return int(f.read().strip())
    except Exception:
        return 0 # soubor neexistuje nebo je poškozený → 0

def uloz_highscore(skore):
    if skore > nacti_highscore():
        with open(HS_FILE, 'w') as f:
            f.write(str(skore))
```

Rekord se zobrazuje v menu (žlutě), ve hře v HUDu (zezelenal při překonání) a na obrazovce Game Over (s textem „Nový rekord!“ pokud byl překonán).

10 Grafické assety

Grafika pochází z volně dostupného balíčku **kenney.nl/assets/space-shooter-remastered** (licence CC0 – veřejná doména). Soubory je potřeba přejmenovat a umístit do složky `assets/` vedle `main.py`.

Přehled požadovaných souborů

Soubor v assets/	Originální název	Použití
lod_1.png	ship_0001.png	Lod' hráče – modrá varianta
lod_2.png	ship_0004.png	Lod' hráče – červená varianta
lod_3.png	ship_0007.png	Lod' hráče – zelená varianta
lod_4.png	ship_0010.png	Lod' hráče – žlutá varianta
meteor_1.png	meteor_large.png	Velký meteorit (typ překážky 0)
meteor_2.png	meteor_small.png	Malý meteorit (typ překážky 0)
nepriteľ.png	enemy_0001.png	Trosky satelitu (typ překážky 1)
vybuch_01.png – vybuch_07.png	effect_*.png	Snímky animace výbuchu (7 ks)
vybuch.png	effect_purple.png	Záložní výbuch (pokud chybí animace)
splash.png	(vlastní obrázek 800×600)	Úvodní obrazovka (5 s)

11 Co by šlo ještě zlepšit

Základní herní mechaniky i vizuální efekty jsou hotové, ale pár věcí by hru mohlo ještě posunout:

- **Podpora myši v menu** – aktuálně funguje menu jen přes klávesnici. Přidat `MOUSEMOTION` pro hover efekt a `MOUSEBUTTONDOWN` pro klik by bylo výrazně user-friendly.

- **Uložení nastavení lodi** – výběr lodi se po zavření hry resetuje. Šlo by ho persistovat do souboru (podobně jako high-score) nebo společně s ním do JSON souboru.
- **Tabulka rekordů** – aktuálně se ukládá jen jedno nejlepší skóre. Top 5 nebo top 10 s možností zadání přezdívky by bylo zajímavé rozšíření.
- **Gamepad podpora** – pygame má `pygame.joystick` modul, takže přidat ovládání přes gamepad by bylo relativně jednoduché.
- **Různé typy levelů** – všechny levely teď vypadají stejně, liší se jen rychlostí a počtem překážek. Mohly by být levely s jiným uspořádáním pruhů nebo speciální události (meteorický déšť, výpadek motorů).
- **Bonusové předměty** – sbírat bonusy (štít, extra život, zpomalení překážek) by přidalo herní hloubku.